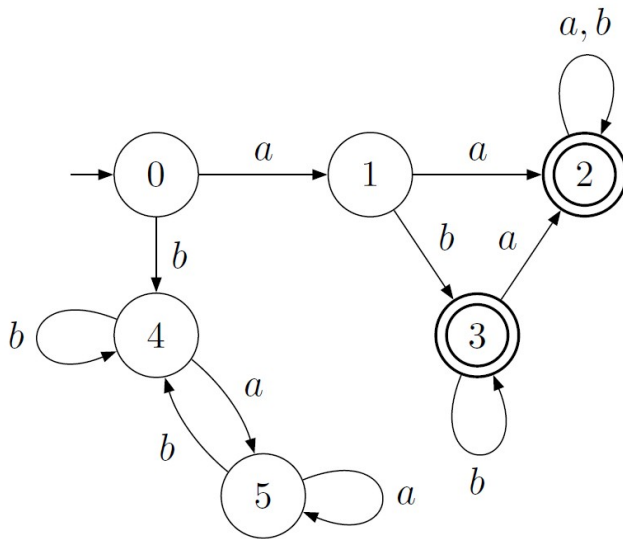




# G52LAC (COMP2040)

## Languages and Computation

### Lecture 5: Regular Expressions (REs) & Equivalence of Regular Expressions and Finite Automata

Tomas Maul  
School of Computer Science  
University of Nottingham Malaysia Campus

# Outline

- Coursework.
- [5 min] Quick recap.
- Regular Expressions.
- Equivalence of Regular Expressions and Finite Automata.

# Outline

- Coursework.
- [5 min] Quick recap.
- Regular Expressions.
- Equivalence of Regular Expressions and Finite Automata.

# Concatenation of Languages (1)

- Given two languages  $L_1, L_2 \subseteq \Sigma^*$ , their concatenation  $L_1 L_2$  is defined by:

$$L_1 L_2 = \{ w_1 w_2 \mid w_1 \in L_1 \wedge w_2 \in L_2 \}$$

- Example:

$$L_1 = \{ \varepsilon, \textit{big}, \textit{small} \}$$

$$L_2 = \{ \textit{cat}, \textit{dog} \}$$

$$\begin{aligned} L_1 L_2 &= \{ w_1 w_2 \mid w_1 \in \{ \varepsilon, \textit{big}, \textit{small} \} \wedge w_2 \in \{ \textit{cat}, \textit{dog} \} \} \\ &= \{ \varepsilon \textit{cat}, \varepsilon \textit{dog}, \textit{bigcat}, \textit{bigdog}, \textit{smallcat}, \textit{smalldog} \} \\ &= \{ \textit{cat}, \textit{dog}, \textit{bigcat}, \textit{bigdog}, \textit{smallcat}, \textit{smalldog} \} \end{aligned}$$

# Concatenation of Languages (2)

- Concatenation of languages is **associative**:

$$L (MN) = (LM) N$$

- Exponent notation is used to denote concatenating a language with itself:

$$L^0 = \{\varepsilon\}$$

$$L^1 = L$$

$$L^2 = LL$$

$$L^3 = LLL$$

$$L^4 = LLLL$$

...

# Kleen Star

Given  $L \subseteq \Sigma^*$ ,  $L^*$  is zero or more concatenations of  $L$ .

$$L^* = \{w_0 w_1 \dots w_{n-1} \mid n, i \in \mathbb{N}, \forall i < n, w_i \in L\}$$

$$L^* = \bigcup_{n=0}^{\infty} L^n = L^0 \cup L^1 \cup L^2 \cup L^3 \cup \dots$$

$$\begin{aligned} & \varepsilon \in L^* \\ w \in L & \Rightarrow w \in L^* \\ w_1 \in L^* \wedge w_2 \in L^* & \Rightarrow w_1 w_2 \in L^* \end{aligned}$$

# Syntax of Regular Expressions

A regular expression is a string made up of alphabet symbols and six RE symbols:  $+$ ,  $*$ ,  $)$ ,  $($ ,  $\emptyset$ ,  $\varepsilon$

- 1  $\emptyset$  is a RE.
- 2  $\varepsilon$  is a RE.
- 3 If  $x \in \Sigma$ , then  $x$  is a RE.
- 4 If  $E$  and  $F$  are REs, then  $E + F$  is a RE.
- 5 If  $E$  and  $F$  are REs, then  $EF$  is a RE.
- 6 If  $E$  is a RE, then  $E^*$  is a RE.
- 7 If  $E$  is a RE, then  $(E)$  is a RE.

# Semantics of Regular Expressions

- The semantics (meaning) of a regular expression is a **set** of words; that is, a language.
- We write  $L(E) = A$  to mean that the language of regular expression  $E$  is the set of words  $A$ .

- 1  $L(\emptyset) = \emptyset$
- 2  $L(\varepsilon) = \{\varepsilon\}$
- 3  $L(x) = \{x\}$  (where  $x \in \Sigma$ )
- 4  $L(E + F) = L(E) \cup L(F)$
- 5  $L(EF) = (L(E))(L(F))$
- 6  $L(E^*) = (L(E))^*$
- 7  $L((E)) = L(E)$



# Notes

- The distinction between:
  - the symbol  $x$  (where  $x \in \Sigma$ ), and
  - the regular expression  **$x$**is often clear from the context.
- In such cases the bold typeface (or underlining when handwritten) is sometimes omitted.

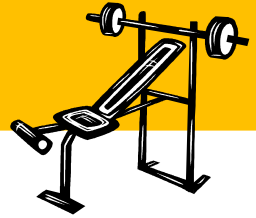
# Relative priority

**Relative priorities:**  $*$  binds more strongly than sequence and  $+$ ; sequence binds more strongly than  $+$ .

As in arithmetic, there are some conventions regarding how to read regular expressions:

- $*$  binds stronger than sequence and  $+$ . E.g. we read  $\mathbf{ab}^*$  as  $\mathbf{a(b^*)}$ . We have to use parentheses to enforce the other reading  $(\mathbf{ab})^*$ .
- Sequencing binds stronger than  $+$ . E.g. we read  $\mathbf{ab + cd}$  as  $(\mathbf{ab}) + (\mathbf{bc})$ . To enforce another reading we have to use parentheses as in  $\mathbf{a(b + c)d}$ .

# Examples / Practice



1. Give examples of words belonging to the languages specified by the following regular expressions.
2. Draw DFAs or NFAs for the following regular expressions:

- $ab$
- $a+b$
- $ab^*$
- $(ab)^*$
- $a+b^*$
- $a+cb^*$
- $(a+b)^*$

Practice. Intuitiveness.

# Recommended Reading

- Introduction to Automata Theory, Languages, and Computation (3rd edition), pages 85–91 & 115–118
- G52MAL Lecture Notes, pages 13–17

# Outline

- Coursework.
- [5 min] Quick recap.
- Regular Expressions.
- Equivalence of Regular Expressions and Finite Automata.

# Objective

- Translating a Regular Expression (RE) into an NFA that accepts precisely the language that the RE describes.
- That is, given an RE  $R$ , construct an NFA  $N$  such that  $L(R) \equiv L(N)$

# Semantics of Regular Expressions

We write  $L(R) = A$  to mean that the language of regular expression  $R$  is the set of words  $A$ .

- 1  $L(\emptyset) = \emptyset$
- 2  $L(\varepsilon) = \{\varepsilon\}$
- 3  $L(x) = \{x\}$  (where  $x \in \Sigma$ )
- 4  $L(R_1 + R_2) = L(R_1) \cup L(R_2)$
- 5  $L(R_1 R_2) = (L(R_1)) (L(R_2))$
- 6  $L(R^*) = (L(R))^*$
- 7  $L((R)) = L(R)$

# Converting Regular Expressions to NFAs

We write  $N(R)$  to mean the NFA that expresses precisely the same language as regular expression  $R$ .

That is,  $L(N(R)) \equiv L(R)$

- ➊  $N(\emptyset)$ : A single initial (and not accepting) state.
- ➋  $N(\varepsilon)$ : A single initial (and accepting) state.
- ➌  $N(x)$ : An initial state, with one transition labelled  $x$  to a second, accepting, state.
- ➍  $N(R_1 + R_2)$ : The NFAs  $N(R_1)$  and  $N(R_2)$  in parallel.
- ➎  $N(R_1 R_2)$ : This is where it gets tricky...



# Sequential Composition of NFAs

$N(R_1 R_2)$ :

- a Place  $N(R_1)$  and  $N(R_2)$  side by side.
- b Remove all initial arrows from  $N(R_2)$ .
- c Any arrows that point to an accepting state in  $N(R_1)$  need to be duplicated to point to **all** states that were initial in  $N(R_2)$ . This includes both transition arrows and initial arrows.
- d All accepting states in  $N(R_1)$  should be changed to not accepting.

**Note:** if you are building  $N(R_1 R_2 R_3)$ , in order to avoid mistakes I would suggest you build things sequentially/carefully, i.e.: first build NFA  $X = N(R_1 R_2)$  then connect  $X$  to  $N(R_3)$ . This is safer than trying to do everything at the same time, by placing  $N(R_1)$ ,  $N(R_2)$  and  $N(R_3)$  next to each other.

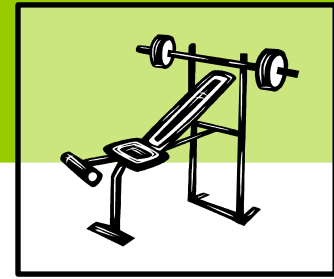
# Repetition of NFAs

6  $N(R^*)$ :

- a Start with  $N(R)$ .
- b All transitions that point to an accepting state need to be duplicated to point to **all** initial states.
- c An additional initial (and accepting) state should be added.

7  $N((R)) = N(R)$

# Examples / Practice



①

$$R1 = a+d$$

$$R2 = b+c$$

$$N(R1)$$

$$N(R2)$$

$$N(R1 \ R2)$$

②

$$R1 = a+cb^*$$

$$R2 = (a+b)^*$$

$$N(R1)$$

$$N(R2)$$

$$N(R1 \ R2)$$

③

$$R = ab$$

$$N(R)$$

$$N(R^*)$$

# Recommended Reading

- G52MAL Lecture Notes, pages 17–25
- Warning: The textbook formulates this translation differently so I suggest you avoid it for this topic.

# Acknowledgements

- Henrik Nilsson.
- Thorsten Altenkirch.
- Neil Sculthorpe.
- Henrik Nilsson.
- V.P. Kallimani.